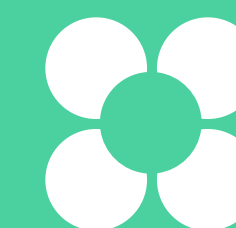
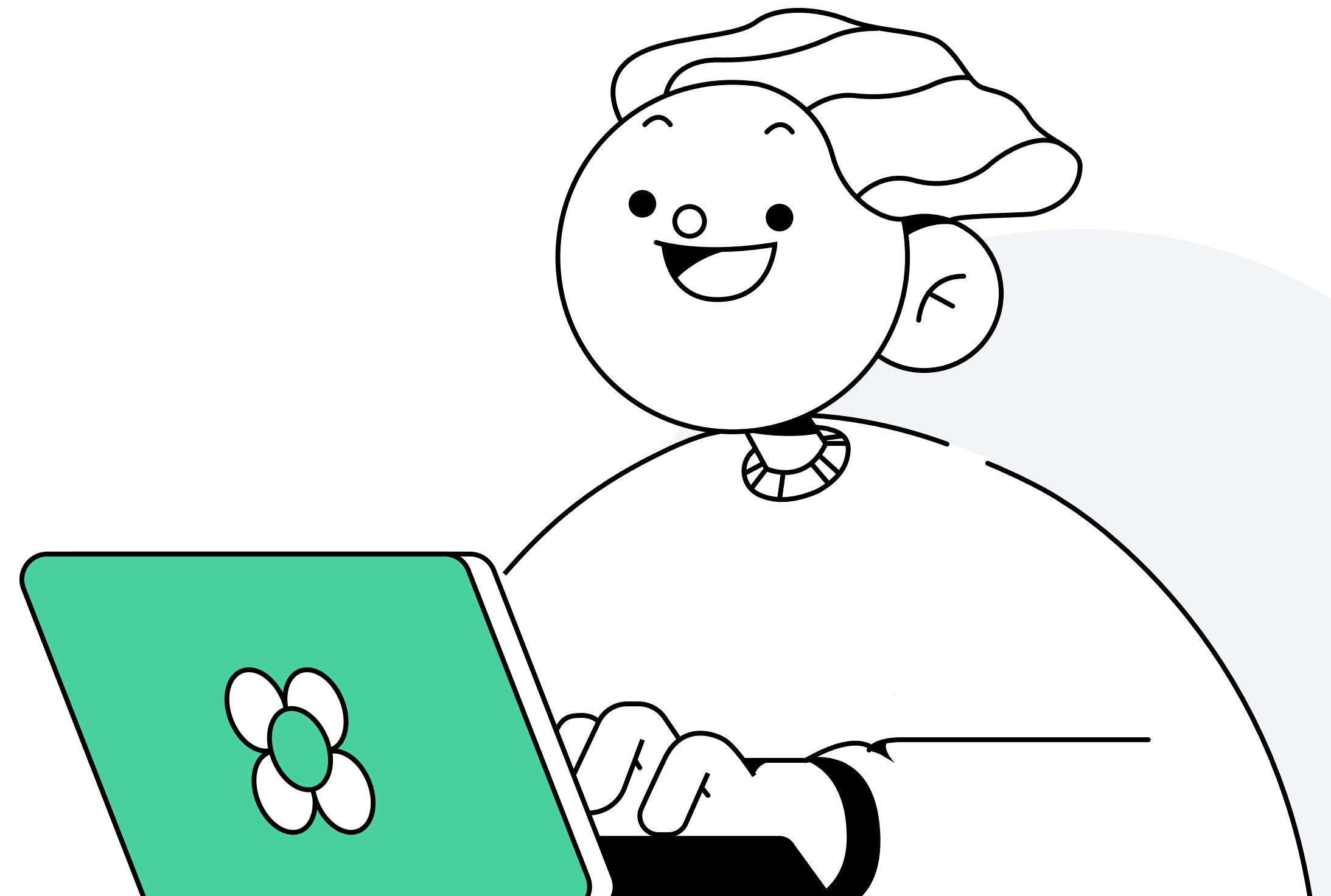


Структура класса



План занятия

- 1 Поля класса
- 2 Методы класса
- 3 Конструкторы



Понятие класса

Для начала познакомимся с термином «парадигма программирования». Это общее понятие, в котором описываются принципы построения и работы программы на конкретном языке.

Парадигм изобретено много, но при разработке на Java обычно сталкиваются с основными тремя:

- 1 **Императивное программирование** описывает шаг за шагом выполняемые действия
- 2 **Функциональное программирование** оперирует функциями, результатами выполнения этих функций и не хранит явно состояние программы
- 3 **Объектно-ориентированное программирование** (ООП) оперирует описанными объектами, которые так или иначе взаимодействуют друг с другом

Понятие класса

Язык Java создавался как объектно-ориентированный язык — это значит, что помимо примитивных типов в языке принято оперировать объектами, приближенными к реальным предметам.

Объект из реального мира «музыкальный трек» обладает характеристиками:

- исполнитель
- альбом
- автор
- продолжительность
- год записи

Понятие класса

Класс — это специальная структура, с помощью которой описываются будущие объекты в Java. Как его применить:

- 1 Создать файл с именем класса и расширением Java
- 2 Внутри этого файла добавить код `class` "Имя файла"

Понятие класса

Пример самого простого пустого класса в файле Record.java:

```
class Record {  
}
```

Специальный оператор `class` говорит компилятору, что начинается описание класса в исходном файле

Описание класса

Рассмотрим более подробное описание класса в Java:

- 1 Каждый класс описывается в пределах фигурных скобок `class Author { ... }`
- 2 Каждый класс может содержать поля — это объявление характеристик объекта
- 3 Поле может быть примитивного и ссылочного типа (любой объект)
- 4 Класс может содержать в себе методы — функции, которые может выполнять объект

Поля класса



Понятие полей

Чтобы хранить необходимую информацию об объекте внутри класса, необходимо создать поля:

```
public class Singer {  
    public String name;  
    public int age;  
    public int rating;  
}
```

Поля — это объявление характеристик объекта. В этом случае полями будут `name`, `age`, `rating`. Эти поля являются **нестатическими**

Имена классов в Java принято писать с заглавной буквы. В нашем случае это `Singer`. Имена классов, переменных, методов должны начинаться с буквы — это требование спецификации языка

Использование статического поля

Когда мы создаём класс со статическим полем, мы создаём одну общую ячейку на программу. К ней можно обратиться через имя класса и название поля:

```
Singer.age = 25;
```

Таким образом статические поля можно использовать как аналог глобальных переменных, которые мы можем применять во всей программе

Использование статического поля

Нестатические поля используются для проектирования памяти объектов. В них указывают, какие данные они должны хранить.

При обращении к нестатической переменной не указывается имя класса, только переменная, в которой находится адрес объекта, содержащего это поле.

```
Singer singer = new Singer();  
singer.name = "Petya";  
singer.age = 8;  
singer.rating = 3;  
f(singer);
```

Необходимо уточнять, к какому именно объекту относится наша переменная. То есть у каждого объекта `Singer` (`singer1`, `singer2`), будет своё поле `name`

Методы класса



Понятие методов класса

Методы класса — это определённые команды, которые проектируют поведение наших объектов.

Рассмотрим предыдущий пример:

```
public class Singer {  
    public String name;  
    public int age;  
    public int rating;  
}
```

Создание методов

Представим, что мы хотим научить объекты нашего певца петь:

```
Singer singer = new Singer ();  
singer.name = "Petya";  
singer.age = 8;  
singer.rating = 3;  
singer.sing ("Good morning");
```

Создание методов

Для этого создаём в классе метод:

```
public class Singer {  
    public String name;  
    public int age;  
    public int rating;  
  
    public void sing(String verse) {  
        System.out.println(Я, “ + name + “, пою тебе: “ + verse);  
    }  
}
```


Понятие полей и методов класса

Чтобы описать метод, нужно объявить:

- Тип возвращаемого значения. В нашем случае это `void`, который говорит, что наш метод ничего не возвращает
- Имя метода. В нашем случае это `sing`
- В круглых скобках передаются входные параметры метода. В нашем случае `String verse`
- Тело метода описывается так же, как и класс, — в фигурных скобках `{...}`

Понятие полей и методов класса

Теперь давайте спросим у певца, не слишком ли он молод:

```
public class Main {  
    public static void main(String[] args) {  
        Singer singer = new Singer ();  
        singer.name = "Petya";  
        singer.age = 8;  
        singer.rating = 3;  
        singer.sing ("Good morning");  
  
        System.out.println(singer.isTooYoung());  
    }  
}
```

Эта команда не будет принимать никаких параметров, но будет возвращать ответ на вопрос: «true» или «false»

Создание методов

Для этого создаём в классе ещё один метод:

```
public class Singer {  
    public String name;  
    public int age;  
    public int rating;  
  
    public void sing(String verse) {  
        System.out.println(Я, “ + name + “, пою тебе: “ + verse);  
    }  
    public boolean isTooYoung() {  
        if (age < 10)  
            return true;  
        else  
            return false;  
    }  
}
```


Прежде чем создавать метод

Ответьте на три вопроса:

- 1 Какое название метода будет хорошо описывать результат его выполнения
- 2 Какое имя входного параметра будет лучше отражать его значение
- 3 Насколько быстро другой разработчик поймёт, что делает наш метод, исходя из выбранных выше значений

Конструкторы



Понятие конструктора

Только что мы научились описывать новые классы, но для работы программы нужно научиться использовать эти классы — создавать из них объекты.

Чтобы создать в Java новый объект из класса, используется оператор `new`. При его вызове в памяти JVM (компьютера/телефона) выделяется область для хранения данных об этом объекте

Понятие конструктора

Ранее мы присваивали каждому полю класса определённое значение:

```
public class Main {  
    public static void main(String[] args) {  
        Singer singer = new Singer ();  
        singer.name = "Petya";  
        singer.age = 8;  
        singer.rating = 3;  
    }  
}
```

Однако всю информацию об объекте можно указать в круглых скобках при создании объекта: `Singer singer = new Singer ();`. Для этого мы можем использовать инструмент «Конструктор».

Конструктор — это специальный метод класса, который ничего не возвращает, но описывает, каким образом должен быть собран/инициализирован класс. Например, какие должны быть значения полей у только что созданного объекта

Понятие конструктора

Если у класса не создать конструктор в описании класса, то компилятор создаст его сам, как произошло в нашем случае с классом `Singer`.

В этом случае все поля, если они не примитивные значения, примут значения `null`

Использование конструктора

Напишем конструктор для класса Singer и попросим его вывести слово «Hello!»:

```
public class Singer {  
    public String name;  
    public int age;  
    public int rating;  
  
    public Singer () {  
        System.out.println("Hello!");  
    }  
}
```

Использование конструктора

Напишем программу так, чтобы при создании певца мы могли передать все необходимые данные. Для этого укажем данные в круглых скобках при создании объекта:

```
public class Main {  
    public static void main(String[] args) {  
        Singer singer = new Singer("Petya", 8, 3);  
    }  
}
```

Использование конструктора

Теперь создадим конструктор, который будет принимать соответствующие параметры:

```
public class Singer {  
    public String name;  
    public int age;  
    public int rating;  
    public Singer () {  
        this.name = name;  
        this.age = age;  
        this.rating = rating;  
    }  
}
```

Переменная `this` указывает на адрес того объекта, у которого вызвали метод или который сейчас проходит стадию конструктора и даёт Java понять, что мы используем поле `name`, а не параметр `name`

Использование конструктора

Java даёт возможность создавать несколько конструкторов. В этом случае вызываться будет только тот конструктор, который подходит под количество, порядок и типы параметров.

Пример: создадим ситуацию, в которой мы не будем сообщать рейтинг певца, и попросим Java автоматически выводить в этом случае рейтинг 3.

Для этого создадим ещё один конструктор с двумя переменными

```
public class Singer {  
    public String name;  
    public int age;  
    public int rating;  
  
    public Singer(String name, int age, int rating) {  
        this.name = name;  
        this.age = age;  
        this.rating = rating;  
    }  
    public Singer(String name, int age) {  
        this(name, age, 3);  
    }  
}
```

Использование статических полей

Введём новый показатель — максимальный рейтинг среди всех певцов. Т. к. максимальный рейтинг — это общий показатель, то будем использовать статическое поле.

`Math.max` — специальная команда, которая вычисляет максимальное число из переданных ей

```
public class Singer {  
    public static int maxRating = 0;  
  
    public String name;  
    public int age;  
    public int rating;  
  
    public Singer(String name, int age, int rating) {  
        this.name = name;  
        this.age = age;  
        this.rating = rating;  
        maxRating = Math.max(maxRating, rating);  
    }  
  
    public Singer(String name, int age) {  
        this(name, age, 3);  
    }  
}
```


Использование статических полей

Теперь создадим второго певца и пропишем команду, которая постоянно будет обновлять максимальный рейтинг и выводить его на экран:

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Макс рейтинг: " + Singer.maxRating);  
  
        Singer singer = new Singer("Petya", 8);  
        System.out.println(singer);  
        System.out.println("Макс рейтинг: " + Singer.maxRating);  
  
        Singer singer2 = new Singer("Anya", 15, 4);  
        System.out.println(singer2);  
        System.out.println("Макс рейтинг: " + Singer.maxRating);  
    }  
}
```